UPSSA
UNIVERSITÉ PRIVÉE DE LA SANTÉ ET DES SCIENCES D'AGADIR
الجامعة الخاصة للصحة و العلوم لأكادير
ⵜⴰⵙⴷⴰⵡⵜ ⵜⵉⵖⵓⵔⵉⵡⵜ ⵏ ⵜⵎⵙⵓⵙⵜ ⴷ ⵜⵜⵓⵖⵔⵉⵡⵉⵏ ⵏ ⵓⴳⴰⴷⵉⵔ

ÉCOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE D'AGADIR
المدرسة العليا الخاصة للهندسة بأكادير
ⵜⵉⵍⵍⵉ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵜⵉⵖⵓⵔⵉⵡⵜ ⵏ ⵓⵜ ⵍⵍⴰⵏ ⴰⴳⴰⴷⵉⵔ

Système d'exploitation Linux

Pr. Salwa BELAQZIZ
s.belaqziz@uiz.ac.ma

Année universitaire 2026-2027

Chapitre 2 : Utilisateurs et permissions

Chaque action est faite au nom d'un compte, identifié par un UID

Le noyau doit savoir **au nom de qui** chaque action est faite pour l'autoriser ou la refuser : il compare l'identité du processus aux droits du fichier.

Utilisateur
Identifié par un **UID**. Possède des fichiers, lance des processus.

→

Groupe
Identifié par un **GID**. Regroupe des utilisateurs qui partagent des droits.

→

Processus
Hérite de l'UID et des GID de celui qui l'a lancé.

→

Fichier
Appartient à un utilisateur et à un groupe.

UID	Type de compte	Exemples
`0`	Super-utilisateur : tous les droits	`root`
`1 – 999`	Comptes système : font tourner les services	`www-data, sshd`
`≥ 1000`	Utilisateurs humains : se connectent	`alice, bob`

Le noyau ne manipule que les **numéros**. Le nom n'est qu'une étiquette lue dans `/etc/passwd`.

root (UID 0) échappe à presque tous les contrôles : on travaille en compte normal et on n'utilise root que pour administrer.

/etc/passwd décrit chaque compte en sept champs

Chaque ligne de `/etc/passwd` décrit un compte, en sept champs séparés par « : ».

ali	x	1001	1001	Ali Salmi	/home/ali	/bin/bash
1 · Identifiant	2 · Mot de passe → /etc/shadow	3 · UID	4 · GID principal	5 · Nom complet (GECOS)	6 · Répertoire personnel	7 · Shell de connexion

Compte	UID	Répertoire	Shell
root	0	/root	/bin/bash
www-data	33	/var/www	/usr/sbin/nologin
mysql	110	/var/lib/mysql	/bin/false
ali	1001	/home/ali	/bin/bash

Shell de connexion

`/bin/bash` : connexion possible.

`nologin` ou `/bin/false` : connexion refusée. C'est le cas des **comptes système**, qui font tourner un service avec le minimum de droits.

```
$ getent passwd ali
$ sudo useradd -r -s /usr/sbin/nologin monapp
```

Lisible par tous (644) : il ne contient aucun mot de passe. On le modifie avec `usermod` ou `vipw`.

/etc/shadow garde les mots de passe hachés, /etc/group décrit les groupes

/etc/shadow · mots de passe hachés (root seul)

```
bob:$y$j9T$Kq…:20454:1:90:7:14:20819:
```

Champ	Signification
1	Identifiant
2	Mot de passe haché (détail ci-dessous)
3	Dernier changement, en jours depuis le 1/1/1970
4 · 5 · 6	Délai mini, durée maxi, préavis (jours)
7 · 8	Délai d'inactivité, expiration du compte

Le champ 2 se lit en trois parties : **algorithme**, **sel**, **hachage**

`$6$`	`Kx9vL2pQ$`	`mT8zQ1rV…`
Algorithme	Sel aléatoire	Hachage

`$1$` MD5 obsolète · `$6$` SHA-512 · `$y$` yescrypt (défaut récent) · `!` en tête : compte verrouillé

/etc/group · les groupes

```
dev:x:1500:alice,bob
```

Champ	Signification
1	Nom du groupe
2	Mot de passe de groupe (rare)
3	GID
4	Membres secondaires

Le groupe **principal** est donné par le champ 4 de `/etc/passwd`, pas par cette liste.

Le sel rend différents les hachages de deux mots de passe identiques : impossible de les comparer à une table précalculée. Un champ vide ne demande aucun mot de passe.

À la connexion, Linux compare le hachage saisi à celui de /etc/shadow

Ce qui se passe quand ali tape son identifiant et son mot de passe :

1 — Saisie
login ou l'écran graphique lit identifiant et mot de passe

→

2 — /etc/passwd
retrouve ali : UID 1001, /home/ali, /bin/bash

→

3 — /etc/shadow
lit le sel et le hachage enregistrés

→

4 — Comparaison
hache le mot de passe saisi avec ce sel et compare

→

5 — Session
si égal : prend l'UID d'ali et lance son shell

Le mot de passe n'est jamais stocké en clair

Le système garde seulement une **empreinte à sens unique**. Pour vérifier, il refait le calcul et compare.

Même root ne peut pas relire un mot de passe : il peut seulement le remplacer.

```
$ sudo tail -3 /var/log/auth.log
sshd: Accepted password for ali
sshd: Failed password for fatima
sshd: Failed password for invalid user admin
# invalid user : le compte n'existe pas
```

id et who disent qui l'on est ; last et les journaux disent qui s'est connecté

Qui suis-je, qui est connecté ?

Commande	Affiche
`whoami`	Le nom de l'utilisateur courant
`id [ali]`	UID, GID principal et tous les groupes
`groups`	La liste des groupes
`who · w`	Les sessions ouvertes (w : leur activité)

```
$ id
uid=1001(ali) gid=1001(ali)
groupes=1001(ali),27(sudo),1500(dev)
$ who
ali   pts/0   2026-10-10 09:12
```

Dans `id`, **gid=** donne le groupe principal et **groupes=** liste tous les groupes, principal compris.

Qui s'est connecté, et quand ?

Commande · fichier	Ce qu'on y apprend
`last -n 5 ali`	Les 5 dernières sessions d'ali
`lastb`	Les tentatives échouées (root)
`lastlog`	Dernière connexion de chaque compte
`faillock --user ali`	Échecs comptés, blocage
`/var/log/auth.log`	Connexions, su, sudo (Debian)
`/var/log/secure`	Même rôle (Red Hat)

```
$ sudo grep 'Failed password' /var/log/auth.log | wc -l
412
```

Un compte utilisé à une heure ou depuis un lieu inhabituels doit alerter : consultez ces journaux régulièrement.

useradd -m crée un compte complet en cinq opérations

Ce que fait `sudo useradd -m -s /bin/bash alice`, étape par étape :

1 **Choisit l'UID**
le premier libre ≥ UID_MIN (1000)

→

2 **Écrit /etc/passwd**
ajoute la ligne d'ali

→

3 **Écrit /etc/shadow**
mot de passe « ! » : compte verrouillé

→

4 **Crée le groupe privé**
alice:x:1001: dans /etc/group

→

5 **Crée /home/ali**
copie le modèle /etc/skel

Les valeurs par défaut viennent de trois emplacements

Emplacement	Ce qu'il fixe
`/etc/login.defs`	Plages d'UID, durée de vie des mots de passe, UMASK
`/etc/default/useradd`	Shell, répertoire de base, expiration
`/etc/skel/`	Modèle copié dans chaque nouveau /home

`useradd -D` affiche souvent `/bin/sh` : pensez à `-s /bin/bash`.

Groupe privé

Chaque utilisateur reçoit un groupe du même nom, dont il est le seul membre : ses fichiers ne sont partagés avec personne par défaut.

Tant que `passwd ali` n'a pas été lancé, personne ne peut se connecter. Modifier les défauts ne touche que les comptes créés **ensuite**.

usermod modifie un compte, userdel le supprime

Commande · option	Effet
`useradd -m`	Crée aussi le répertoire personnel
`-s /bin/bash · -c "Ali Ahmed"`	Shell · nom complet
`-g dev · -G web,sudo`	Groupe principal · secondaires
`-u 1050 · -e 2027-06-30`	UID imposé · date d'expiration
`usermod -aG dev ali`	Ajoute ali au groupe dev
`usermod -l fafa fatima`	Renomme le compte
`usermod -d /data/ali -m ali`	Déplace le répertoire personnel
`userdel ali`	Supprime le compte, garde /home
`userdel -r ali`	Supprime le compte et son /home

```
$ sudo useradd -m -s /bin/bash \
    -c "Ali Ahmed" -G dev ali
$ sudo passwd ali
$ sudo usermod -aG sudo ali
```

Le piège de -G : `usermod -G web bob` **remplace** la liste des groupes secondaires. Ali perd sudo et dev.

Toujours écrire `usermod -aG` (a = append).

Sur Debian et Ubuntu, `adduser ali` fait la même chose en mode interactif.

passwd change le mot de passe, chage règle sa durée de vie

Commande	Effet
`passwd`	Change son propre mot de passe
`passwd ali`	root change celui de ali
`passwd -e ali`	Force un changement à la prochaine connexion
`passwd -S ali`	Affiche l'état du mot de passe
`chage -l ali`	Affiche les règles d'expiration
`chage -M 90 ali`	Mot de passe valable 90 jours au plus
`chage -m 1 -W 7 ali`	1 jour mini entre deux changements, préavis 7 jours
`chage -E 2027-01-01 ali`	Le compte expire à cette date

```
$ sudo chage -l ali
Dernier changement   : janv. 01, 2026
Mot de passe expire  : avril 01, 2026
Mot de passe inactif : avril 15, 2026
Le compte expire     : janv. 01, 2027
Nb max. de jours     : 90
Avertissement        : 7
```

Expiré ou inactif ?

Mot de passe expiré : bob se connecte encore, mais doit en choisir un nouveau.

Compte inactif ou expiré : la connexion est refusée ; seul root peut le réactiver.

Verrouiller un mot de passe ne bloque pas toutes les connexions

Bloquer un compte peut viser le mot de passe, le shell ou le compte entier. Le bon choix dépend de ce qui doit rester possible

Commande	Ce qu'elle change	Clé SSH	Annuler
`passwd -l ali`	Ajoute ! devant le hachage	Encore possible	`passwd -u ali`
`usermod -L ali`	Identique à passwd -l	Encore possible	`usermod -U ali`
`usermod -s /usr/sbin/nologin ali`	Remplace le shell de connexion	Bloquée	`usermod -s /bin/bash ali`
`chage -E 0 ali`	Fait expirer le compte	Bloquée	`chage -E -1 ali`

```
$ sudo passwd -S ali
ali L 2026-01-01 1 90 7 14   # L = verrouillé
```

Le groupe principal sert à créer, les groupes secondaires à accéder

Groupe principal

Un seul, champ 4 de `/etc/passwd`.

Sert à **créer** : chaque nouveau fichier prend ce groupe.

Groupes secondaires

Zéro ou plusieurs, listés dans `/etc/group`.

Servent à **accéder** : ils comptent comme le principal.

Commande	Effet
`groupadd -g 1500 dev`	Crée dev
`groupmod -n devs dev`	Renomme
`groupdel dev`	Supprime
`gpasswd -a ali dev`	Ajoute ali
`gpasswd -d ali dev`	Retire ali
`gpasswd -A ali dev`	ali gère dev

```
$ id ali
uid=1002(ali) gid=1002(ali) groupes=1002(ali),1500(dev)
$ touch rapport.txt ; ls -l rapport.txt
-rw-r--r-- 1 ali ali 0 rapport.txt   # ali:ali, pas dev
```

Obtenir le groupe dev sur un fichier : `chgrp dev` après , `newgrp dev` pour la session, ou le **SGID** sur le répertoire .

Un nouveau membre doit **se reconnecter** (ou lancer `newgrp dev`) pour profiter du groupe.

Un compte vit quatre étapes : arrivée, mutation, absence et départ

1 · Arrivée

Créer le compte, forcer le changement du mot de passe

```
$ useradd -m -G dev ali
$ passwd ali
$ passwd -e ali
```

2 · Mutation

Changer d'équipe sans perdre les autres groupes

```
$ usermod -aG compta ali
$ gpasswd -d ali dev
```

3 · Absence

Bloquer temporairement, puis rétablir

```
$ usermod -L ali
$ usermod -U ali
```

4 · Départ

Bloquer, archiver, puis supprimer

```
$ chage -E 0 ali
$ tar czf ali.tgz /home/ali
$ userdel -r ali
```

Pourquoi archiver avant de supprimer ?

`userdel -r` efface /home/ali, mais les fichiers de lea ailleurs (/srv, /tmp) restent avec un UID devenu **orphelin** : `ls -l` affiche un numéro à la place du nom.

Pour les retrouver : `sudo find / -nouser -o -nogroup 2>/dev/null`

su ouvre une session sous une autre identité, sudo exécute une seule commande

La syntaxe de su a été vue au chapitre 1. Ici, on compare les deux façons d'agir avec d'autres droits.

Critère	su · substitute user	sudo · superuser do
Portée	Un shell complet sous une autre identité	Une seule commande
Mot de passe demandé	Celui du compte cible (root)	Le sien, mémorisé 15 min
Ce qui est permis	Tout ce que peut le compte cible	Seulement ce qu'autorise /etc/sudoers
Traçabilité	On sait qu'on est devenu root, pas ce qu'on a fait	Chaque commande est journalisée avec son auteur

```
$ sudo apt update  # une commande en root
$ sudo -u www-data ls /var/www  # en tant qu'un autre compte
$ sudo -i  # shell root, à éviter
$ sudo -l  # mes droits sudo
$ sudo !!  # relancer la dernière commande avec sudo
```

Une règle sudoers précise qui peut lancer quoi, sous quelle identité

Chaque règle de `/etc/sudoers` répond à quatre questions :

`ali`	`ALL`	`(ALL:ALL)`	`/usr/bin/apt`
Qui ? un utilisateur ou %groupe	**Sur quelle machine ?** ALL = toutes	**En tant que qui ?** utilisateur:groupe cible	**Quelles commandes ?** ALL ou chemins absolus

```
# /etc/sudoers.d/web  (sudo visudo -f …)
%sudo   ALL=(ALL:ALL) ALL      # Debian, Ubuntu
%wheel  ALL=(ALL) ALL          # Red Hat
User_Alias WEBADM = ali, fatima
Cmnd_Alias WEB = /usr/bin/systemctl restart nginx, \
                 /usr/bin/systemctl reload nginx
WEBADM  ALL=(root) WEB
%dev    ALL=(www-data) /usr/bin/git pull
```

Élément	Sens
`User_Alias`	Liste nommée d'utilisateurs
`Cmnd_Alias`	Liste nommée de commandes
`%dev`	Les membres du groupe dev
`NOPASSWD:`	Pas de mot de passe

Toujours éditer avec `visudo`, qui vérifie la syntaxe : une erreur dans sudoers bloque sudo pour tout le monde.

sudo est sûr quand chaque compte n'a que les commandes dont il a besoin

Pratique	Pourquoi
Travailler en compte normal	Une faute de frappe en root peut détruire le système
Préférer sudo à su	Chaque commande est tracée et attribuée
Lister les commandes permises	ALL donne root sans restriction
Écrire des chemins absolus	Un faux apt placé dans le PATH serait accepté
Éviter NOPASSWD	Un poste resté ouvert donne root à qui passe
Une règle par fichier dans sudoers.d	Facile à relire et à retirer

```
$ sudo journalctl \
   _COMM=sudo | tail -1
ali : TTY=pts/0 ;
USER=root ;
COMMAND=/usr/bin/apt
update
```

`sudo vi` ou `sudo bash` ouvre un shell root complet : la règle « une commande précise » est contournée.

Ne donnez jamais un éditeur ou un shell par sudo.

chown change le propriétaire d'un fichier, chgrp change son groupe

Qui a le droit ?

Changer le **propriétaire** : root seulement.

Changer le **groupe** : le propriétaire, vers un groupe dont il est membre.

Commande	Effet
`chown ali f`	ali devient propriétaire
`chown ali:dev f`	Propriétaire et groupe
`chown :dev f`	Groupe seulement
`chgrp dev f`	Groupe seulement
`chown -R ali:dev projet/`	Tout le contenu du répertoire

`chown` et `chgrp` ne touchent pas aux droits : ils changent **à qui** s'appliquent les blocs u et g.

Pour modifier les droits eux-mêmes, on utilise `chmod` (chapitre 1).

Le noyau teste le propriétaire, puis le groupe, puis les autres, et s'arrête au premier qui correspond



Une seule catégorie

La **première** qui correspond s'applique, et le test s'arrête.

Avec `----rw----`, le propriétaire n'a **aucun** droit, même s'il est dans le groupe.

Supprimer un fichier dépend des droits du **répertoire** (w + x), pas de ceux du fichier.

Sur un répertoire, x ouvre le passage et r, w n'agissent qu'avec lui

Le rôle de r, w et x sur un répertoire est vu au chapitre 1. Ce qui compte en pratique, c'est leur **combinaison** :

Droits	ls rep	cd rep	cat rep/f	touch rep/n
`r--`	noms seuls [1]	non	non	non
`--x`	non	oui	oui [2]	non
`r-x`	oui	oui	oui	non
`-wx`	non	oui	oui [2]	oui
`rwx`	oui	oui	oui	oui

[1] ls affiche les noms avec des « ? » à la place des détails. [2] Seulement si l'on connaît le nom exact du fichier.

w sans x ne sert à rien : pour créer ou supprimer dans un répertoire, il faut w et x.

`chmod 711` sur son répertoire personnel : les autres traversent vers un fichier connu, sans pouvoir lister le contenu.

Quatre valeurs de umask couvrent presque tous les besoins

le umask **retire** des droits à 666 (fichier) et 777 (répertoire). Il n'en ajoute jamais.

umask	Fichier créé	Répertoire créé	Usage typique
`022`	`644 rw-r--r--`	`755 rwxr-xr-x`	Défaut : tout le monde lit
`002`	`664 rw-rw-r--`	`775 rwxrwxr-x`	Travail en équipe avec un groupe partagé
`027`	`640 rw-r-----`	`750 rwxr-x---`	Serveur : rien pour les autres
`077`	`600 rw-------`	`700 rwx------`	Données personnelles sensibles

Rendre le réglage durable

```
$ umask 027  # ce shell seulement
$ echo 'umask 027' >> ~/.bashrc  # un utilisateur
$ grep ^UMASK /etc/login.defs  # tous les comptes
UMASK 022
```

Lire le umask en clair

`umask -S` affiche les droits **conservés** : u=rwx,g=rx,o=rx pour 022.

Un fichier n'est jamais créé exécutable : il faut un `chmod +x`.

Exercice 1 : Lire une ligne de /etc/passwd

Voici la ligne du compte de ali dans le fichier `/etc/passwd`.

```
# /etc/passwd (une ligne)
ali:x:1002:1002:ali Ahmed:
  /home/bob:/bin/bash
```

Questions

Q1. Quel est le **nom de connexion** de ali et son **UID** ?

Q2. ali est-il un utilisateur humain ou un compte système ?

Q3. Quel est son **répertoire personnel** ?

Q4. ali peut-il ouvrir une session ? Où est rangé son mot de passe ?

Aide

Les 7 champs sont séparés par « : » : identifiant, mot de passe, UID, GID, commentaire, répertoire, shell.

Exercice 1 : Lire une ligne de /etc/passwd

Q1. Nom : ali · UID : 1002

Le 1er champ est le nom de connexion, le 3eme champ est l'UID.

Q2. Un utilisateur humain

Son UID vaut 1002, donc au moins 1000. Les comptes système ont un UID de 1 à 999.

Q3. /home/ali

C'est le 6e champ : le répertoire où ali arrive quand il se connecte.

Q4. Oui · mot de passe dans /etc/shadow

Son shell (7eme champ) est /bin/bash, un vrai shell. Le x indique que le mot de passe haché est dans /etc/shadow, lisible par root seul.

À retenir

Le shell indique si la connexion est possible : /bin/bash l'autorise, /usr/sbin/nologin la refuse.

Exercice 2 : Lire la sortie de la commande id

Ali tape la commande `id` dans son terminal et obtient le résultat ci-contre. Le système est une Ubuntu.

Questions

Q1. Quel est l'**UID** d'Ali ?

Q2. Quel est son **groupe principal** ?

Q3. Quels sont ses **groupes secondaires** ?

Q4. Ali peut-elle utiliser `sudo` ? Pourquoi ?

```
$ id
uid=1001(ali) gid=1001(ali)
  groupes=1001(ali),27(sudo),
  1500(dev)
```

Aide

gid= donne le groupe principal. groupes= liste tous les groupes, le principal compris.

Exercice 2 : Lire la sortie de la commande id

Q1. UID : 1001

On le lit après uid= : 1001, avec le nom ali entre parenthèses.

Q2. Groupe principal : ali (GID 1001)

C'est la valeur après gid=. Chaque utilisateur reçoit un groupe privé du même nom.

Q3. Groupes secondaires : sudo et dev

Ce sont les groupes de la liste groupes=, sauf le groupe principal alice.

Q4. Oui

Sur Debian et Ubuntu, les membres du groupe sudo peuvent lancer des commandes en root avec sudo. Sur Red Hat, c'est le groupe wheel.

À retenir

id résume l'identité d'un utilisateur : son UID, son groupe principal et tous ses groupes.

Exercice 3 : Créer un compte utilisateur

Une nouvelle étudiante, Sara, arrive. Vous êtes administrateur et devez lui créer un compte.

Questions

Q1. Écrivez la commande qui crée le compte **sara** avec un répertoire personnel et le shell `/bin/bash`.

Q2. Quelle commande lui donne un **mot de passe** ?

Q3. Comment **vérifier** que le compte existe ?

Q4. Que se passe-t-il si on oublie de donner un mot de passe ?

```
# options utiles de useradd
-m   crée le répertoire personnel
-s   choisit le shell
-c   nom complet
```

Aide

Les commandes d'administration se lancent avec sudo.

Exercice 3 : Créer un compte utilisateur

Q1. sudo useradd -m -s /bin/bash sara

-m crée /home/sara et y copie les fichiers de /etc/skel. -s /bin/bash choisit le shell de connexion.

Q2. sudo passwd sara

root saisit deux fois le nouveau mot de passe. Il est enregistré haché dans /etc/shadow.

Q3. getent passwd sara ou id sara

Si la commande affiche une ligne pour sara, le compte existe.

Q4. Personne ne peut se connecter

Sans mot de passe, le compte est verrouillé : /etc/shadow contient « ! » à la place du hachage.

```
$ sudo useradd -m -s /bin/bash sara
$ sudo passwd sara
Nouveau mot de passe :
$ id sara
uid=1003(sara) gid=1003(sara)
  groupes=1003(sara)
```

À retenir

Créer un compte se fait en deux temps : useradd crée le compte, passwd l'active.

Exercice 4 : Ajouter un utilisateur à un groupe

Fatima est déjà membre du groupe **dev**. Elle doit aussi entrer dans le groupe **web**, sans quitter dev.

Questions

Q1. Écrivez la commande qui ajoute Fatima au groupe **web**.

Q2. Que se passe-t-il si on oublie l'option `-a` ?

Q3. Comment **vérifier** les groupes de Fatima ?

Q4. À quel moment Fatima profite-t-il de son nouveau groupe ?

```
$ groups Fatima
Fatima dev
```

Aide

-G donne la liste des groupes secondaires. -a veut dire « ajouter » à la liste.

Exercice 4 : Ajouter un utilisateur à un groupe

Q1. sudo usermod -aG web Fatima

-aG ajoute web à la liste des groupes de Fatima et garde les groupes qu'il avait déjà.

Q2. Fatima perd le groupe dev

Sans -a, -G remplace toute la liste : Fatima n'a plus que web. On peut aussi utiliser gpasswd -a Fatima web.

Q3. groups Fatima ou id Fatima

La liste doit maintenant contenir dev et web.

Q4. À sa prochaine connexion

Les groupes sont lus à l'ouverture de session. Fatima se déconnecte et se reconnecte, ou tape newgrp web.

```
$ sudo usermod -aG web fatima
$ groups fatima
fatima dev web
  # avec -G seul (erreur) :
$ sudo usermod -G web fatima
$ groups fatima
fatima web
```

À retenir

Toujours écrire usermod -aG : le -a évite d'effacer les autres groupes.

Exercice 5 : Choisir entre su et sudo

Ali est membre du groupe sudo. Il tape les deux commandes ci-contre.

```
$ sudo apt update
$ su -
```

Questions

Q1. Quel **mot de passe** demande `sudo apt update` ?

Q2. Quel mot de passe demande `su -` ?

Q3. Laquelle des deux méthodes **enregistre** chaque commande dans le journal ?

Q4. Comment Ali revient-il à son compte après `su -` ?

Aide

sudo lance une seule commande. su ouvre une session complète sous une autre identité.

Exercice 5 : Choisir entre su et sudo

Q1. Le mot de passe d'Ali

sudo vérifie qu'Ali est bien Ali, puis consulte /etc/sudoers pour savoir s'il a le droit.

Q2. Le mot de passe de root

su ouvre une session root : il faut connaître le mot de passe du compte cible.

Q3. sudo

Chaque commande lancée avec sudo est enregistrée avec le nom d'Ali. Avec su, on sait seulement qu'une session root a été ouverte.

Q4. Avec la commande exit

exit ferme la session root et ramène Ali dans son propre shell.

```
$ su -
Mot de passe : (celui de root)
# whoami
root
# exit
$ whoami
ali
```

À retenir

On préfère sudo : on ne partage pas le mot de passe root et chaque action est tracée.

Exercice 6 : Qui peut lire ce fichier ?

Fatima est propriétaire de notes.txt. Ali est membre du groupe **dev**. Ahmed n'est pas dans dev.

Questions

Q1. Que peut faire **Fatima** sur ce fichier ?

Q2. Que peut faire **Ali** ?

Q3. Que peut faire **Ahmed** ?

Q4. Ali peut-il **modifier** le fichier ?

```
$ ls -l notes.txt
-rw-r----- 1 fatima dev 900
  notes.txt
```

Aide

Le système teste d'abord le propriétaire, puis le groupe, puis les autres. Il s'arrête au premier cas qui correspond.

Exercice 6 : Qui peut lire ce fichier ?

Q1. Fatima : lire et modifier

Alice est la propriétaire, donc le 1er bloc s'applique : rw-.

Q2. Ali : lire seulement

Bob n'est pas propriétaire mais il est membre de dev : le 2eme bloc s'applique, r--.

Q3. Ahmed : rien

Ahmed n'est ni propriétaire ni membre de dev : le 3eme bloc s'applique, ---.

Q4. Non

Le bloc du groupe ne contient pas w. Pour l'autoriser, Alice tape chmod g+w notes.txt.

```
-rw-r----- 1 fatima dev notes.txt
  # rw-  propriétaire (fatima)
  # r--  groupe dev (ali)
  # ---  autres (ahmed)
```

À retenir

Une seule catégorie s'applique à chaque personne : la première qui correspond.

SUID, SGID et sticky bit changent l'identité d'exécution ou la règle de suppression

SUID · u+s · 4000

Sur un **programme** : il s'exécute avec l'UID de son **propriétaire**.

Ex. : `/usr/bin/passwd` (root) écrit dans /etc/shadow.

SGID · g+s · 2000

Sur un **répertoire** : les fichiers créés dedans prennent **son groupe**.

Sur un programme : il s'exécute avec le groupe du fichier.

Sticky · o+t · 1000

Sur un **répertoire partagé** : seul le propriétaire d'un fichier peut le supprimer.

Ex. : `/tmp` (drwxrwxrwt).

```
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 59976 /usr/bin/passwd
$ sudo chmod g+s /srv/equipe
$ ls -ld /srv/equipe
drwxrwsr-x 2 root dev 4096 /srv/equipe
```

L'octal passe à quatre chiffres

2	7	7	0
spéciaux	u	g	o

`chmod 2770` : groupe hérité · `chmod 1777` : comme /tmp · `chmod 4755` : SUID

Le **s** ou le **t** remplace le x dans `ls -l` ; une **majuscule** (S, T) signale un bit posé sans x. Un programme SUID root mal écrit donne root à tous ; Linux ignore le SUID sur les scripts.

Le SUID donne au processus l'UID effectif du propriétaire du programme

Chaque processus porte deux identités : l'**UID réel** (qui l'a lancé) et l'**UID effectif** (dont les droits sont testés). Le SUID ne change que l'UID effectif.

ali lance passwd
UID réel : ali

→

Processus passwd
réel : ali
effectif : **root**

→

/etc/shadow
rw-r----- root

→

Écriture autorisée
l'UID effectif est root

```
$ passwd &  # ali, en arrière-plan
$ ps -o pid,ruser,euser,cmd -C passwd
  PID RUSER  EUSER  CMD
  4210 ali  root   passwd
$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1450 /etc/shadow
```

Pourquoi passwd reste sûr

Il lit l'**UID réel** pour savoir qui l'appelle : ali ne change que son propre mot de passe.

Il demande l'ancien mot de passe avant tout changement.

Un répertoire d'équipe combine groupe, SGID, sticky bit et umask

Besoin : ali et sara (groupe dev) partagent /srv/equipe. Chacun modifie les fichiers de l'autre, personne d'autre n'entre, et nul ne supprime le fichier d'un collègue.

1
Créer le groupe
`groupadd dev ; usermod -aG dev ali`

→

2
Créer le répertoire
`mkdir /srv/equipe`

→

3
Donner le groupe
`chgrp dev /srv/equipe`

→

4
Droits, SGID, sticky
`chmod 3770 /srv/equipe`

→

5
umask 002
fichiers créés modifiables par le groupe

```
$ ls -ld /srv/equipe
drwxrws--T 2 root dev 4096 /srv/equipe
$ touch /srv/equipe/plan.txt  # ali
$ ls -l /srv/equipe
-rw-rw-r-- 1 ali dev 0 plan.txt
$ rm /srv/equipe/plan.txt  # sara
rm: Opération non permise
```

Lire 3770

3 = 2 (SGID) + 1 (sticky)

770 : rwx pour root et dev, rien pour les autres

s dans le groupe : héritage de dev

T : sticky posé, autres sans x

Les ACL donnent des droits à un utilisateur ou un groupe précis

Les droits classiques ne connaissent que trois catégories. Pour donner la lecture à **ali seul**, sans l'ajouter au groupe, on ajoute une entrée nommée : une ACL (Access Control List).

Commande	Effet
`setfacl -m u:ali:r f`	Donne r à ali
`setfacl -m g:compta:rw f`	Donne rw au groupe compta
`setfacl -x u:ali f`	Retire l'entrée de ali
`setfacl -b f`	Supprime toute l'ACL
`setfacl -R -m g:audit:rX rep`	Toute l'arborescence
`getfacl f`	Affiche l'ACL

rX : x seulement sur les répertoires, pour donner la lecture d'un dossier entier.

```
$ setfacl -m u:ali:r rapport.txt
$ ls -l rapport.txt
-rw-r-----+ 1 ali dev 812 rapport.txt
$ getfacl rapport.txt
user::rw-
user:ali:r--
group::r--
mask::r--
other::---
```

Le **+** à la fin des droits dans `ls -l` signale qu'une ACL existe.

Le mask d'une ACL plafonne les droits effectifs

Le **mask** fixe le maximum accordé aux utilisateurs nommés, aux groupes nommés et au groupe propriétaire. Le droit effectif est l'intersection de l'entrée et du mask.

Entrée	Accordé	mask	Effectif
user:ali	`rwx`	`r--`	`r--`
group::	`r-x`	`r--`	`r--`
user:: (proprio)	`rw-`	—	`rw-`
other::	`---`	—	`---`

```
$ setfacl -m u:ali:rwx rapport.txt
$ setfacl -m m::r rapport.txt
$ getfacl rapport.txt
user::rw-
user:ali:rwx   #effective:r--
group::r--
mask::r--
other::---
```

Pourquoi le mask change tout seul

Chaque `setfacl -m` le recalcule pour couvrir toutes les entrées. Un `chmod g=r` modifie en réalité le mask.

Avec une ACL, la colonne **groupe** de `ls -l` montre le mask, pas les droits du groupe.

Une ACL par défaut fait hériter ses droits ; cp -a et tar --acls les conservent

Hériter : l'ACL par défaut (-d)

Posée sur un répertoire, elle est copiée sur chaque fichier ou sous-répertoire créé dedans. Elle remplace le umask pour ces fichiers.

```
$ setfacl -d -m g:dev:rwx /srv/projet
$ getfacl /srv/projet
group::rwx
default:group:dev:rwx
default:mask::rwx
```

Les fichiers **déjà présents** ne changent pas : on les traite avec `setfacl -R -m`. `setfacl -k` retire l'ACL par défaut.

Conserver : copier et sauvegarder

Commande	Effet
`cp -a f /sauvegarde/`	Copie avec les ACL
`tar --acls -czf a.tgz rep`	Archive avec les ACL
`getfacl -R rep > acl.txt`	Sauvegarde les ACL
`setfacl --restore=acl.txt`	Les restaure

Un `cp` simple ou un `tar` sans `--acls` **perd les ACL** : la copie n'a plus que les droits classiques.

Un groupe suffit pour une équipe ; une ACL sert pour les exceptions

Critère	Groupe + droits classiques	ACL
Nombre d'acteurs	Un propriétaire, un groupe, les autres	Autant d'utilisateurs et de groupes que nécessaire
Lisibilité	Visible d'un coup d'œil avec ls -l	Il faut getfacl pour tout voir
Support	Tous les systèmes de fichiers et outils	ext4, xfs, btrfs ; cp et tar demandent des options
Gestion	Ajouter quelqu'un = l'ajouter au groupe	Une règle par personne, sur chaque fichier
Cas typique	Une équipe stable qui partage un dossier	Une exception : un auditeur en lecture seule

Règle pratique : commencez par un groupe et le SGID. Ajoutez une ACL seulement pour l'exception qu'un groupe ne sait pas exprimer.

chattr protège un fichier, même contre root

Les attributs du système de fichiers (ext4, xfs) s'ajoutent aux droits et s'appliquent **même à root** : ils protègent contre les erreurs et les modifications malveillantes.

Attribut · commande	Effet
`i`	Immuable : ni modification, ni suppression, ni renommage
`a`	Ajout seul : écrire à la fin, sans réécrire ni supprimer
`chattr +i f`	Pose l'attribut (root seul)
`chattr -i f`	Le retire
`lsattr f`	Affiche les attributs

Usage typique : `+a` sur un journal, pour qu'on ne puisse pas effacer ses traces.

```
$ sudo chattr +i /etc/resolv.conf
$ lsattr /etc/resolv.conf
----i---------e---- /etc/resolv.conf
$ sudo rm /etc/resolv.conf
rm: Opération non permise
$ sudo chattr +a /var/log/app.log
$ echo ok >> /var/log/app.log  # accepté
```

Si root obtient « Opération non permise », pensez à `lsattr` : un attribut **i** en est la cause la plus fréquente.

« Permission non accordée » se diagnostique en cinq vérifications

Quand une commande renvoie « Permission non accordée », vérifiez dans cet ordre :

1. **Qui suis-je ?** · `id` : utilisateur et groupes actifs. Groupe ajouté récemment ? Se reconnecter.
2. **Chaque répertoire du chemin** · `namei -l /chemin/f` : il faut x sur chaque répertoire traversé.
3. **Propriétaire et groupe** · `ls -l` : quelle catégorie (u, g ou o) s'applique à moi ?
4. **Droit demandé** · Lire : r · modifier : w · exécuter : x · supprimer : w+x sur le répertoire.
5. **ACL et attributs** · `getfacl` (mask) et `lsattr` (attribut i).

```
$ cat /srv/compta/budget.ods
cat: Permission non accordée
$ namei -l /srv/compta/budget.ods
drwxr-xr-x root root  /
drwxr-xr-x root root  srv
drwxr-x--- root dev   compta
-rw-r----- alice dev  budget.ods
$ id
uid=1003(carla) …  # pas dans dev
```

Le blocage est sur **compta** : carla relève des « autres », qui n'ont aucun droit.

Synthèse du chapitre 2 · utilisateurs et permissions

Partie	À retenir
1 · Identités	UID et GID ; root = 0, comptes système 1–999, humains ≥ 1000 ; /etc/passwd (7 champs, lisible par tous), /etc/shadow (hachage salé, root seul), /etc/group
2 · Comptes et groupes	useradd -m, usermod -aG (jamais -G seul), userdel -r ; passwd, chage -M -E ; groupe principal pour créer, secondaires pour accéder
3 · su et sudo	su - : session sous une autre identité ; sudo : une commande, son propre mot de passe, tracée ; règles via visudo
4 · Contrôle d'accès	chown (root), chgrp ; test dans l'ordre u, g, o ; x indispensable sur un répertoire ; umask 022, 002, 027, 077
5 · Aller plus loin	SUID 4, SGID 2 (héritage du groupe), sticky 1 ; ACL : setfacl, getfacl, mask, -d ; chattr +i / +a ; diagnostic en 5 étapes

Exercice 7 : Reconnaître les droits spéciaux

L'administrateur affiche les trois éléments ci-contre avec `ls -ld`.

Questions

Q1. Quel droit spécial porte **/usr/bin/passwd** ? À quoi sert-il ?

Q2. Quel droit spécial porte **/tmp** ?

Q3. Quel droit spécial porte **/srv/equipe** ?

Q4. Dans /tmp, Ali peut-il supprimer un fichier du propriétaire Sara ?

```
$ ls -ld /usr/bin/passwd /tmp \
  /srv/equipe
-rwsr-xr-x root root  passwd
drwxrwxrwt root root  /tmp
drwxrws--- root dev   equipe
```

Cherchez la lettre qui remplace le x : s dans le bloc propriétaire, s dans le bloc groupe, t dans le bloc autres.

Exercice 7 : Reconnaître les droits spéciaux

Q1. SUID (s dans le bloc propriétaire)

passwd s'exécute avec les droits de root, son propriétaire. Il peut ainsi écrire le nouveau mot de passe dans /etc/shadow.

Q2. Sticky bit (t dans le bloc autres)

Tout le monde peut écrire dans /tmp, mais chacun ne peut supprimer que ses propres fichiers.

Q3. SGID (s dans le bloc groupe)

Chaque fichier créé dans /srv/equipe prend le groupe dev, et non le groupe de son créateur.

Q4. Non

Le sticky bit l'interdit : seul le propriétaire du fichier, ou root, peut le supprimer.

```
$ rm /tmp/fichier_de_sara
rm: impossible de supprimer :
  Opération non permise
  # SUID 4 · SGID 2 · sticky 1
```

SUID : identité du propriétaire. SGID : groupe hérité.
Sticky : chacun supprime seulement ses fichiers.